

7. Архитектура и проектирование

7.1. Параллельные прямые функционала и реализации

«Параллельные прямые не пересекаются, сколько бы их не продолжали». Из геометрии

В предыдущих рассуждениях о бизнес- и системной аналитике все ясно: мы стремимся дать более-менее адекватное описание того, как программная система будет представлять окружающий мир и как она будет вписываться в происходящие в нем процессы. Процесс реализации (кодирование) – тоже более-менее понятно: берешь и пишешь код, как умеешь. Между этими «что должна делать система» и «как это можно сделать» лежит архитектурная пропасть (рис.7-1), преодолеть которую – задача архитектуры. В унифицированном процессе для этой цели предусмотрена отдельная технологическая дисциплина – **проектирование или программный дизайн** (design, не путать с графическим дизайном), концентрированным выражением которого является архитектура.

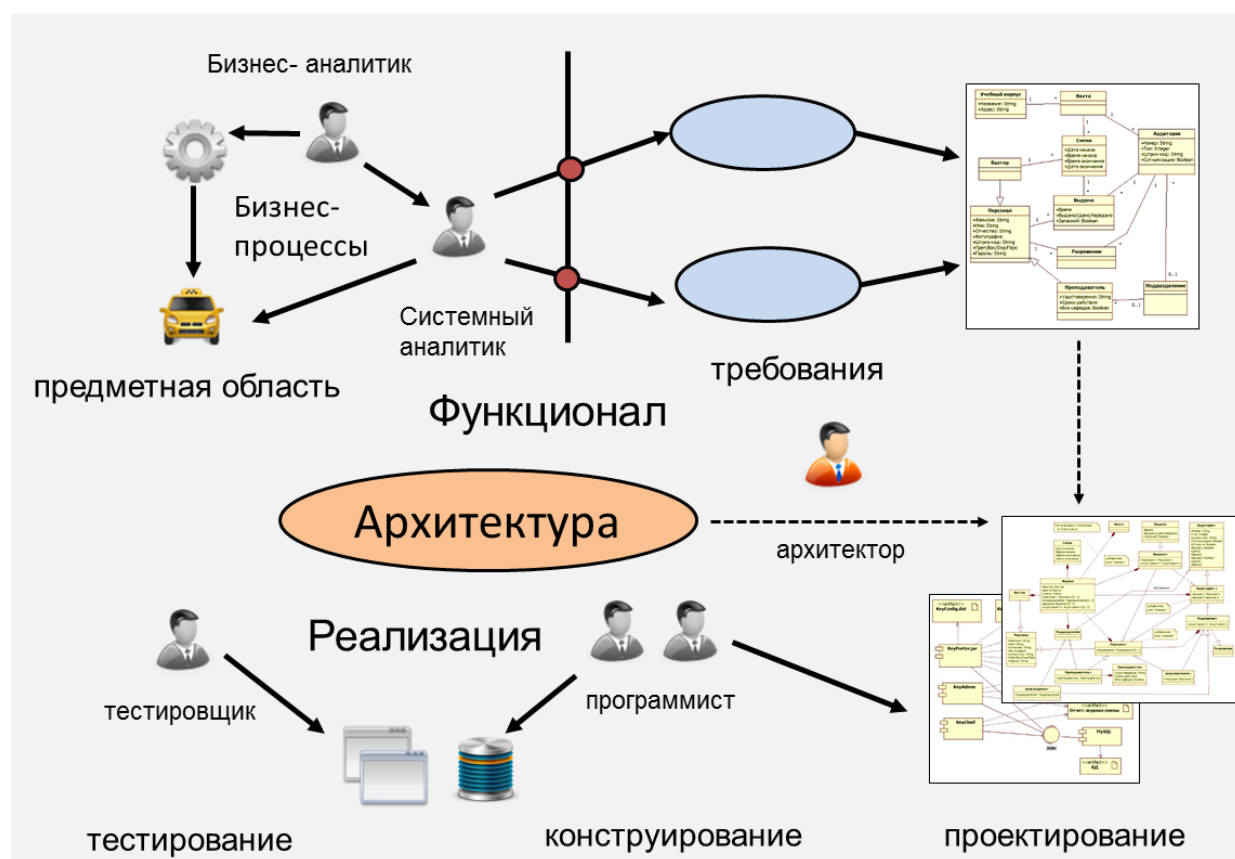


Рис.7-1. Архитектурная пропасть между функционалом и реализацией

Что говорят классики

Наиболее распространенное определение архитектуры [7-1] включает в это понятие:

- **значимые решения** по поводу организации ПС;
- **структурные элементы** и их интерфейсы, при помощи которых конструируется система;

- **поведение** - взаимодействие между этими элементами;
- **компоновка** элементов в иерархию подсистем;
- **стиль** архитектуры, который направляет эту организацию.

Казалось бы, все просто и понятно. Опиши систему на основе приведенных правил, и дело с концом. Однако на практике приходилось видеть и такие описания (рис 7-2).

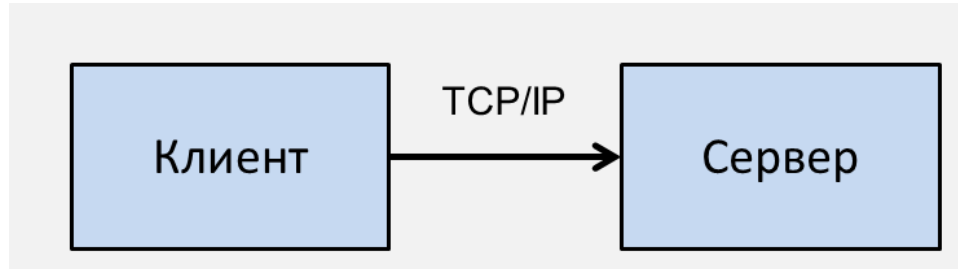


Рис.7-2. «Архитектура» клиент-серверного приложения

Кто скажет, что это не архитектура, пусть первым бросит в меня камень. По крайней мере, первые два пункта формально выполнены. Так все-таки чего же в ней не хватает? И почему разработчики не в состоянии двинуться дальше?

Принципы: разумная достаточность, существенность, множественность представления

Если взять любую сложную техническую систему, например, самолет, то можно нарисовать картинку, подобную приведенной выше, например, контур с крыльями, хвостовым оперением и иллюминаторами. Можно, наоборот, дать полную спецификацию всех деталей и узлов - несколько сотен тысяч. И то и другое не будет **архитектурой**. Первое – по недостаточности, второе – по той же причине, что и полный исходный текст программы не дает понимания того, как она работает. Для архитектуры важны другие аспекты.

Сущность системы, разумная достаточность - описание, достаточное для понимания ее сущности и процессов функционирования, излишние подробности столь же вредны, как и пробелы.

Множественность представления – система не имеет единственного представления, ее описание состоит из нескольких представлений (аспектов, проекций, срезов), которые только в реализации создадут единое целое. Классический перечень представлений, именуемый «4+1», содержит [7-1]:

- **функциональное** представление – сценарии, прецеденты, требования, предметная область;
- **логическое** представление – представление системы с точки зрения конечного пользователя, внешний вид, логическая структура;
- **процедурное** представление – параллелизм, потоки, взаимодействие, масштабируемость, производительность;
- представление **развертывания** – компоновка, топология, коммуникации, администрирование, настройка;
- представление **разработки** – структура программного кода, библиотеки, интерфейсы, абстракции.

Ключевым и объединяющим представлением является функциональное (рис.7-3). Здесь вопросы возникают только к логическому представлению: представление системы с точки зрения конечного пользователя, внешний вид, логическая структура. Как раз представление конечного пользователя не является решающим, он может иметь какие угодно предубеждения относительно внутреннего устройства системы. Под логическим представлением логичнее понимать **согласованное для всех заинтересованных сторон представление о реализации функциональности на архитектурной модели проекта**, т.е. понимание как функциональное представление системы реализуется в архитектуре, понятное всем, имеющим отношение к проекту.

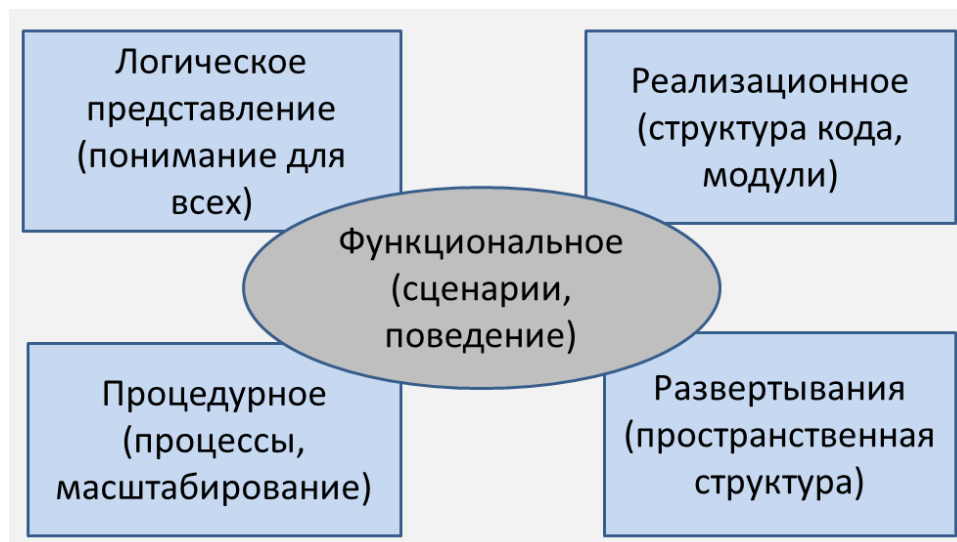


Рис.7-3.Представления архитектуры в модели «4+1»

Архитектура – минимально избыточное, согласованное описание системы, включающее структуру, поведение, компоновку, стили разработки и значимые решения, создающее адекватное представление о ключевых моментах ее организации для всех заинтересованных сторон

Термин *архитектура* в описанном виде относится к ПО системы в целом. Но он применим также в бизнес- и системной аналитике, где также необходимо собственное согласованное описание, поэтому бывают различные архитектуры и архитекторы со своими видами деятельности:

- **бизнес-архитектура** – бизнес-цели проекта, обоснование его успешности, экономические, социальные и прочие аспекты разработки ПС, доказательства того, что проект «выстрелит»;
- **системная архитектура** – описание системы, создаваемой преимущественно из готовых компонент, интеграция, адаптация под бизнес-процессы;
- **архитектура ПО** – описание системы со значительной долей разрабатываемого кода, взгляд на программный продукт *изнутри*.

Требования к архитекторам ПО

Так почему же разработка архитектуры не продвигается дальше представленной на рис.7-2? Ответ прост. Требуется знание «матчасти» на соответствующем уровне, т.е. минимально избыточном, но необходимо подробном. Сюда входят: известные архитектурные компоненты, решения и стандарты, фреймворки, шаблоны проектирования, архитектурные стили, внутренние проблемы функционирования

программного кода и способы их решения. Попросту говоря, архитектуру не из чего собирать, а ее проблемы не известны (рис.7-4).

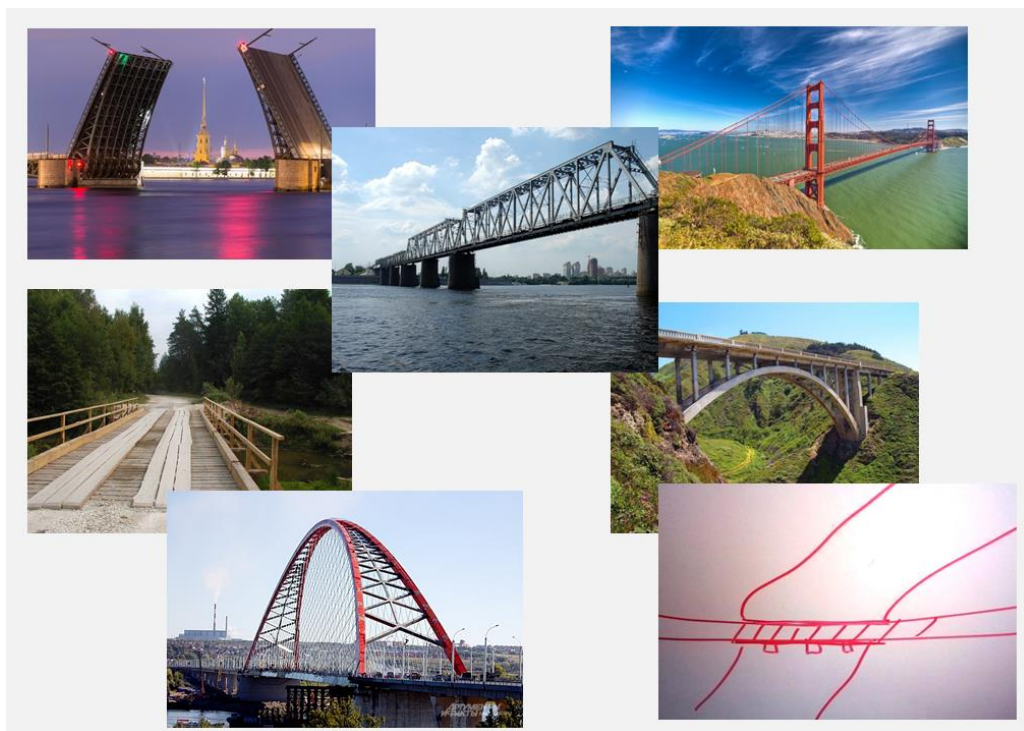


Рис.7-4. Программная архитектура и представление дилетанта

С другой стороны, архитектор не должен *изобретать велосипед*, он, как правило, использует уже известные идеи, его деятельность можно охарактеризовать как *искусство создания оригинального продукта из доступных компонент и решений*.

И последнее. Профессия архитектора, как и профессия руководителя, опирается на обобщенный практический опыт. Образно говоря, *архитекторами не рождаются, ими становятся*.

Ниже приведена выборка из описания трудовых функций профессионального стандарта «Архитектор программного обеспечения» [1-1]:

- разработка решений для повторного использования компонентов;
- определение перечня возможных слоев программных компонентов;
- определение перечня возможных протоколов взаимодействия компонентов;
- определение перечня возможных шаблонов (стилей) проектирования для каждого слоя или компонента;
- анализ качества кода: анализ зависимостей, статический анализ кода;
- оценка и выбор модели управления исключениями.

Естественно, что специалист без навыков и опыта работы с кодом не может принимать перечисленные решения. В то же время ему свойственны функции руководителя и управленца. В [7-2] перечислены требования к архитектору в области информационных технологий:

- технические знания – пресловутая «матчасть»:
 - ИТ-стратегия;
 - Enterprise Architecture;
 - методологии проектирования и ведения проектов (UML, TOGAF, MSF, OUM);

- интеграционная архитектура (SOA и EDA);
 - управление бизнес-процессами (BPM);
 - процессы управления IT-инфраструктурой (ITIL/ITSM);
 - архитектура и функциональность приложений (IC, Oracle, SAP, IBM WebSphere);
 - архитектура инфраструктурных сервисов (LDAP, MS Exchange, DNS, DHCP, ...);
 - архитектура сети (TCP/IP, VOIP).;
 - архитектура центров обработки данных (электропитание, кондиционирование, пожаротушение, оборудование);
 - облачные вычисления и виртуализация;
 - фреймворки;
 - методологии, стандарты и лучшие практики;
 - языки и среды разработки: Java, .NET, SQL, PL/SQL, Delphi;
 - операционные системы Windows, Linux, IBM AIX, Solaris.
- технические навыки – деятельности, связанные с управлением требованиями, проектированием и управлением программными проектами:
 - сбор и управление требованиями;
 - умение оценивать время и стоимость решения;
 - проектирование систем;
 - умение писать документы;
 - умение работать на высоком уровне абстракции;
 - управление техническими рисками;
 - следование методологиям;
 - умение создавать решения без личных предпочтений в технологиях.
 - бизнес и менеджмент. Архитектор должен понимать: по каким законам функционирует бизнес. На чем компании зарабатывают деньги. Как при помощи ИТ создавать ценность для бизнеса. Чем отличаются OPEX от CAPEX. И что предпочтительнее для конкретного бизнеса. Как управлять проектами. Как управлять рисками. Как управлять людьми. Как оценивать время и стоимость решения. Отраслевые стандарты. Предметные области, в которых ведутся проекты.
 - личные навыки (Soft Skills): переговоры, манипуляции, английский язык, управление конфликтами, продажи, проектная деятельность, дипломатия (политика и интриги), ораторское искусство, умение брать ответственность, умение передавать ответственность, умение отстаивать свою позицию, тайм-менеджмент, самопрезентация.

Архитектурные аспекты проектирования

Согласно SWEBOK [1-3] большинство аспектов проектирования имеют прямое отношение к архитектуре, т.к. представляют собой *значимые решения*, принятые в организации системы:

- ключевые идеи и концепции: абстрагирование, связность и соединение, модульность и декомпозиция, инкапсуляция, разделение интерфейса и реализации;
- параллелизм;
- контроль и обработка событий;
- распределение компонентов;
- обработка ошибок и исключительных ситуаций и обеспечение отказоустойчивости;
- взаимодействие и представление (шаблон MVC);
- сохраняемость данных - доступность «долгоживущих» данных;
- виды представления архитектуры - структурное, поведенческое, логическое, физическое, реализация кода;
- архитектурные стили;
- шаблоны проектирования;
- методы проектирования: нисходящее проектирование, модульное, абстрагирование, итеративность, функционально-ориентированное или структурное проектирование, объектно-ориентированное проектирование, проектирование на основе структур данных, компонентное проектирование.

Замечание по теме. Что будет, если какой-либо ключевой момент, например, устойчивость, способность к восстановлению, не будет отражен в архитектуре. В этом случае в процессе кодирования возникающие проблемы будут решаться по принципу «как получится», что может существенно ухудшить характеристики системы, а при накоплении таких решений поставить предел ее развитию. Выходом из такой ситуации будет только глубокий реинжиниринг кода - *развалить* систему и собрать заново.